

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра автоматизованих систем управління



Звіт

до лабораторної роботи №2
з дисципліни:

“Паралельні обчислення і розподілені системи”

на тему:

**“RESTful сервіси (частина 1). Розробка базового REST API
у Spring Boot ”**

Виконав: студент ОІ-31
Джебко Юрій

Прийняв:
Скорохода О. В.

Львів – 2026

Лабораторна робота №2

RESTful сервіси (частина 1). Розробка базового REST API у Spring Boot

Мета роботи: сформована базова доменна модель та API, які можуть бути масштабовані.

Хід роботи

1. Ознайомитися з принципами REST-архітектури
2. Обрати предметну область

Логістична компанія

Сутності: Відправлення, Склад, Маршрут, Транспорт, Клієнт, Статус.

Бізнес-логіка:

- оптимізація маршруту;
- відстеження вантажу;
- зміна складу зберігання;
- формування логістичних звітів.

Мікросервіси:

- Shipment Service;
- Warehouse Service;
- Routing Service;
- Tracking Service.

3. Спроекувати:

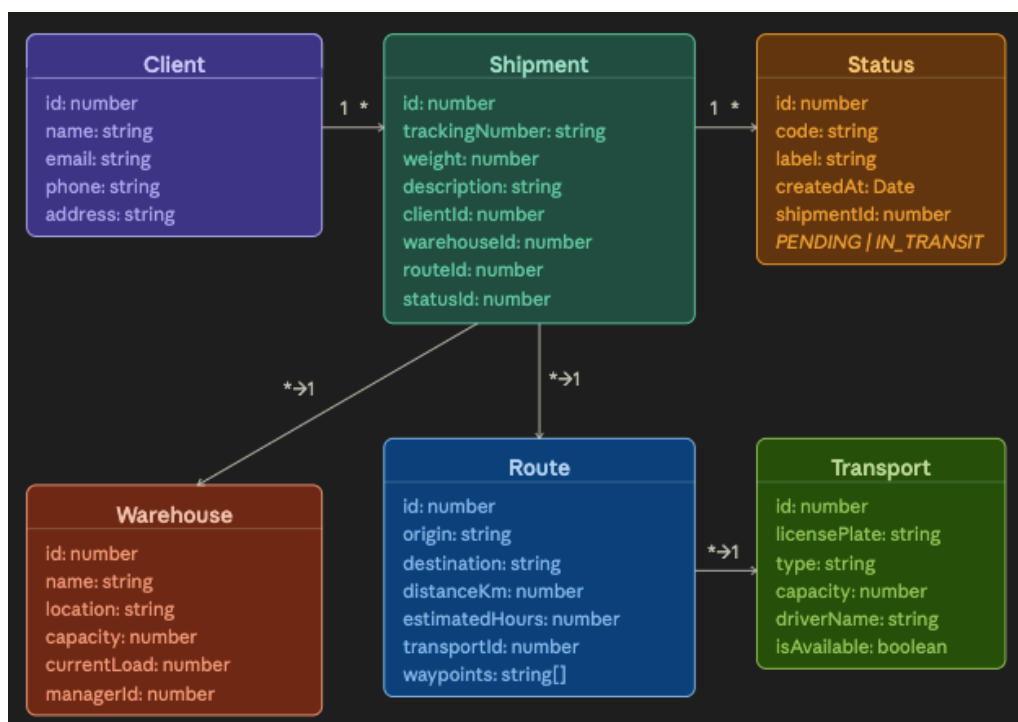


Рис.1. UML-діаграма класів

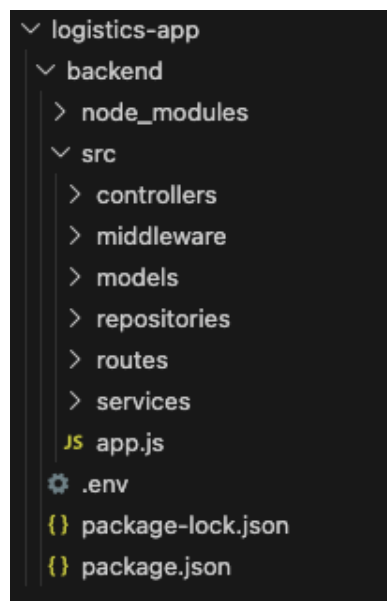
REST-ендпоінти:

Метод	URL	Призначення	Коди відповіді
/api/clients			
GET	/api/clients	Отримати список усіх клієнтів	200
GET	/api/clients/:id	Отримати клієнта за ID	200 / 404
POST	/api/clients	Створити нового клієнта	201 / 400 / 409
PUT	/api/clients/:id	Оновити дані клієнта	200 / 404 / 409
DELETE	/api/clients/:id	Видалити клієнта	204 / 404 / 409
/api/shipments			
GET	/api/shipments	Отримати список усіх відправлень	200
GET	/api/shipments/:id	Отримати відправлення за ID	200 / 404
GET	/api/shipments/track/:tracking	Відстежити вантаж за номером	200 / 404
POST	/api/shipments	Створити нове відправлення	201 / 400 / 409
PUT	/api/shipments/:id	Оновити відправлення	200 / 404
DELETE	/api/shipments/:id	Видалити відправлення	204 / 404
/api/warehouses			
GET	/api/warehouses	Отримати список усіх складів	200
GET	/api/warehouses/:id	Отримати склад за ID	200 / 404
POST	/api/warehouses	Створити новий склад	201 / 400

PUT	/api/ warehouses/:id	Оновити дані складу	200 / 404
DELETE	/api/ warehouses/:id	Видалити склад	204 / 404 / 409
POST	/api/ warehouses/ transfer	Перемістити посилку між складами	200 / 404 / 409
/api/routes			
GET	/api/routes	Отримати всі маршрути	200
GET	/api/routes/:id	Отримати маршрут за ID	200 / 404
POST	/api/routes/ optimize	Оптимізувати маршрут	200 / 400 / 404
POST	/api/routes	Створити новий маршрут	201 / 400
PUT	/api/routes/:id	Оновити маршрут	200 / 400 / 404
DELETE	/api/routes/:id	Видалити маршрут	204 / 404
/api/transport			
GET	/api/transport	Отримати всі транспортні засоби	200
GET	/api/ transport/:id	Отримати транспорт за ID	200 / 404
GET	/api/transport/ available	Отримати вільні транспортні засоби	200
POST	/api/transport	Додати транспортний засіб	201 / 400
PUT	/api/ transport/:id	Оновити транспорт	200 / 400 / 404
DELETE	/api/ transport/:id	Видалити транспорт	204 / 404 / 409
/api/statuses			
GET	/api/statuses	Отримати всі статуси	200

POST	/api/statuses	Додати новий статус до відправлення	201 / 400 / 409
GET	/api/statuses/ shipment/:id	Отримати всі статуси конкретного відправлення	200 / 404
GET	/api/statuses/ shipment/:id/ latest	Отримати поточний статус конкретного відправлення	200 / 404

4. Структура проєкту:



5. Реалізація:

Приклад моделі Shipment.js:

```

class Shipment {
  constructor({ id, trackingNumber, weight, description, clientId, warehouseId,
routeId }) {
    this.id = id;
    this.trackingNumber = trackingNumber || `TRK-${Date.now()}-${id}`;
    this.weight = weight;
    this.description = description || null;
    this.clientId = clientId;
    this.warehouseId = warehouseId || null;
    this.routeId = routeId || null;
    this.createdAt = new Date().toISOString();
  }
}

module.exports = Shipment;

```

Приклад контролера Shipment.js:

```

const service = require('../services/shipmentService');
const { validationResult } = require('express-validator');

```

```

module.exports = {
  getAll: (req, res) => {
    res.json(service.getAll());
  },

  getById: (req, res, next) => {
    try {
      res.json(service.getById(req.params.id));
    } catch (e) { next(e); }
  },

  trackByNumber: (req, res, next) => {
    try {
      res.json(service.trackByNumber(req.params.trackingNumber));
    } catch (e) { next(e); }
  },

  create: (req, res, next) => {
    try {
      const errors = validationResult(req);
      if (!errors.isEmpty()) return res.status(400).json({ errors:
errors.array() });
      res.status(201).json(service.create(req.body));
    } catch (e) { next(e); }
  },

  update: (req, res, next) => {
    try {
      res.json(service.update(req.params.id, req.body));
    } catch (e) { next(e); }
  },

  remove: (req, res, next) => {
    try {
      service.remove(req.params.id);
      res.status(204).send();
    } catch (e) { next(e); }
  }
};

```

Приклад сервиса Shipment.js:

```

const repo = require('../repositories/shipmentRepository');
const clientRepo = require('../repositories/clientRepository');
const warehouseRepo = require('../repositories/warehouseRepository');
const routeRepo = require('../repositories/routeRepository');
const statusRepo = require('../repositories/statusRepository');

module.exports = {
  getAll: () => {
    return repo.findAll().map(s => ({
      ...s,
      currentStatus: statusRepo.findLatestByShipmentId(s.id),
    }));
  },

  getById: (id) => {
    const shipment = repo.findById(id);
    if (!shipment) throw { status: 404, message: `Shipment with id=${id} not
found` };
    return {
      ...shipment,
      currentStatus: statusRepo.findLatestByShipmentId(id),
      statusHistory: statusRepo.findByShipmentId(id),
    };
  }
};

```

```

    };
  },

  trackByNumber: (trackingNumber) => {
    const shipment = repo.findByTrackingNumber(trackingNumber);
    if (!shipment) {
      throw { status: 404, message: `Tracking number "${trackingNumber}" not
found` };
    }
    return {
      ...shipment,
      currentStatus: statusRepo.findLatestByShipmentId(shipment.id),
      statusHistory: statusRepo.findByShipmentId(shipment.id),
    };
  },

  create: (data) => {
    if (!clientRepo.exists(data.clientId)) {
      throw { status: 400, message: `Client with id=${data.clientId} does not
exist` };
    }

    if (data.warehouseId) {
      const warehouse = warehouseRepo.findById(data.warehouseId);
      if (!warehouse) {
        throw { status: 400, message: `Warehouse with id=${data.warehouseId}
does not exist` };
      }
      if (warehouse.currentLoad >= warehouse.capacity) {
        throw { status: 409, message: `Warehouse id=${data.warehouseId} is at
full capacity` };
      }
    }

    if (data.routeId && !routeRepo.exists(data.routeId)) {
      throw { status: 400, message: `Route with id=${data.routeId} does not
exist` };
    }

    const shipment = repo.create(data);

    statusRepo.create({ shipmentId: shipment.id, code: 'PENDING', note:
'Shipment created' });

    return { ...shipment, currentStatus:
statusRepo.findLatestByShipmentId(shipment.id) };
  },

  update: (id, data) => {
    if (!repo.exists(id)) throw { status: 404, message: `Shipment with id=${id}
not found` };
    return repo.update(id, data);
  },

  remove: (id) => {
    if (!repo.exists(id)) throw { status: 404, message: `Shipment with id=${id}
not found` };
    repo.remove(id);
  },
};

```

Приклад репозиторія Shipment.js:

```
const Shipment = require('../models/Shipment');

let shipments = [];
let nextId = 1;

module.exports = {
  findAll: () => [...shipments],

  findById: (id) => shipments.find(s => s.id === Number(id)) || null,

  findByTrackingNumber: (trackingNumber) =>
    shipments.find(s => s.trackingNumber === trackingNumber) || null,

  findByClientId: (clientId) =>
    shipments.filter(s => s.clientId === Number(clientId)),

  findByWarehouseId: (warehouseId) =>
    shipments.filter(s => s.warehouseId === Number(warehouseId)),

  create: (data) => {
    const shipment = new Shipment({ id: nextId++, ...data });
    shipments.push(shipment);
    return shipment;
  },

  update: (id, data) => {
    const idx = shipments.findIndex(s => s.id === Number(id));
    if (idx === -1) return null;
    shipments[idx] = { ...shipments[idx], ...data, id: Number(id) };
    return shipments[idx];
  },

  remove: (id) => {
    const idx = shipments.findIndex(s => s.id === Number(id));
    if (idx === -1) return false;
    shipments.splice(idx, 1);
    return true;
  },

  exists: (id) => shipments.some(s => s.id === Number(id)),
};
```

Обробник помилок ErrorHandler.js:

```
module.exports = (err, req, res, next) => {
  const status = err.status || 500;

  res.status(status).json({
    timestamp: new Date().toISOString(),
    status: status,
    message: err.message || 'Internal Server Error',
    path: req.originalUrl
  });
};
```

Валідатор validate.js

```
const { validationResult } = require('express-validator');

module.exports = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
```



```

return res.status(400).json({
  timestamp: new Date().toISOString(),
  status: 400,
  message: 'Validation failed',
  path: req.originalUrl,
  details: errors.array()
});
}
next();
};

```

6. Тестування:

Тестовий сценарій №1: Клієнти

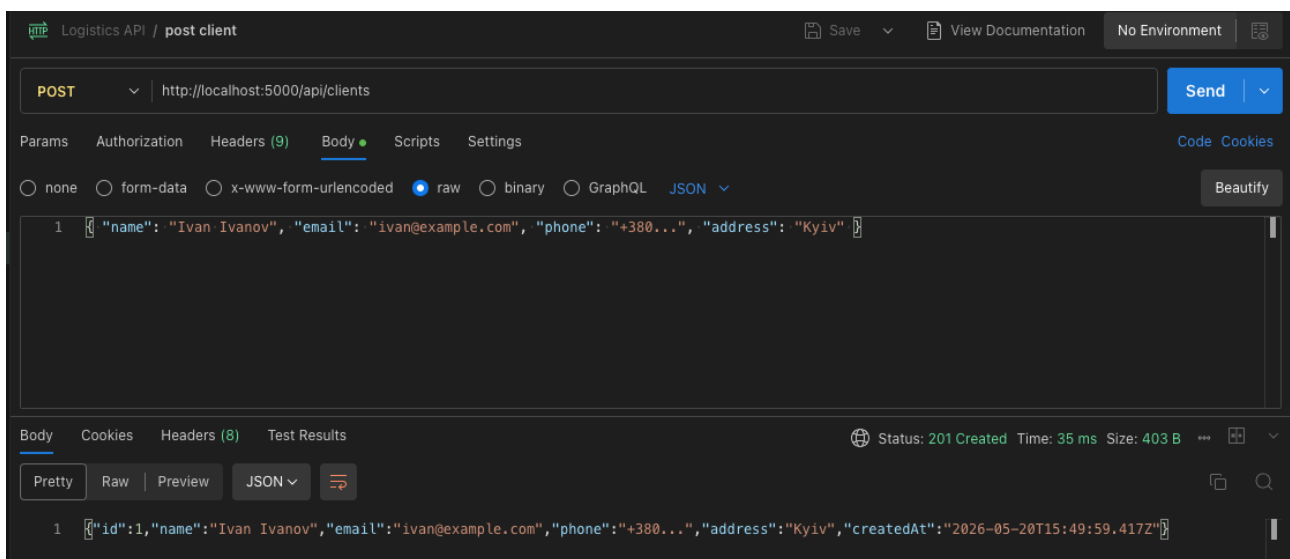


Рис. 1. POST-запит для клієнтів.

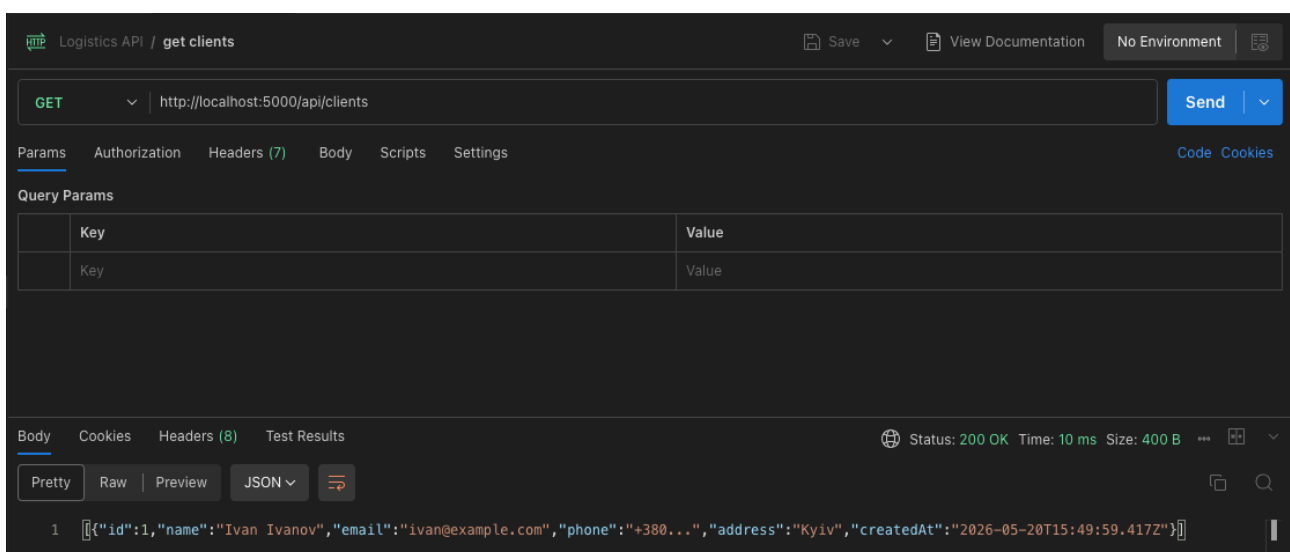


Рис. 2. GET-запит для клієнтів. Виводить дані які ми раніше передали.

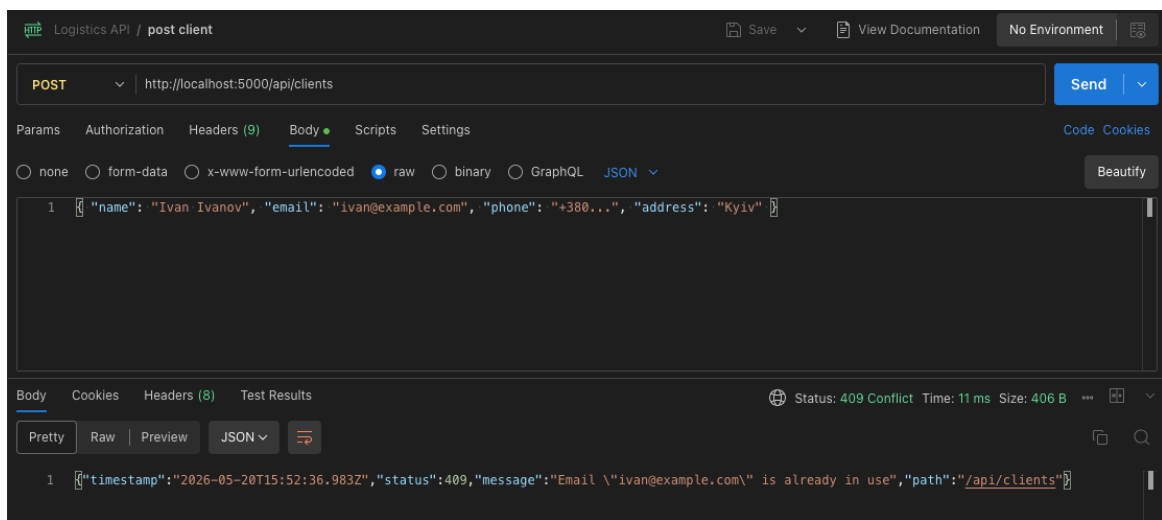


Рис. 3. Повторний POST-запит для клієнтів. Видає помилку, бо такий користувач уже існує.

Тестовий сценарій №2: Створення відправлення

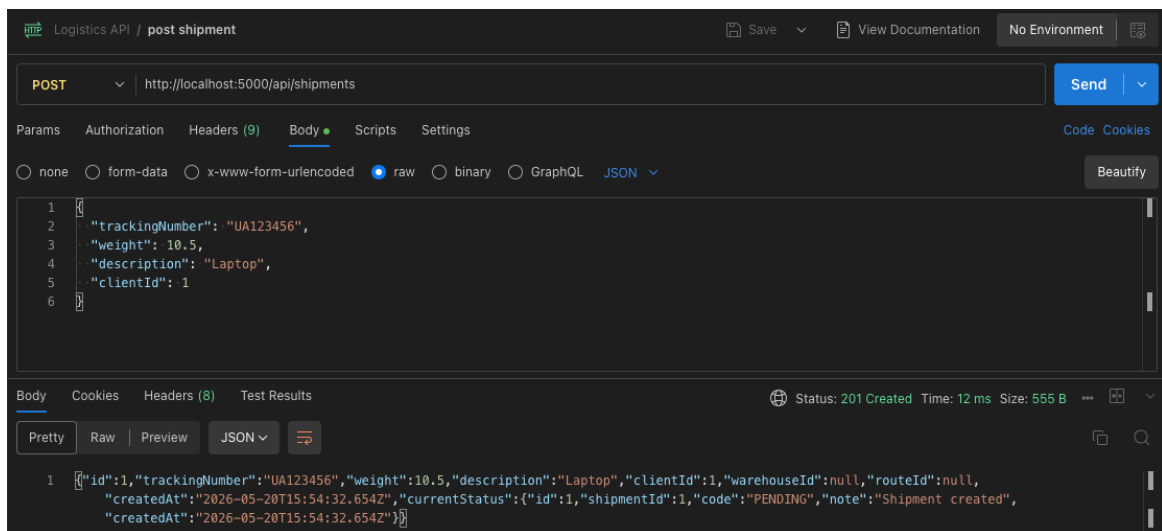


Рис. 4. POST-запит для відправлень. При створенні автоматично додається статус PENDING.

Тестовий сценарій №3: Статуси

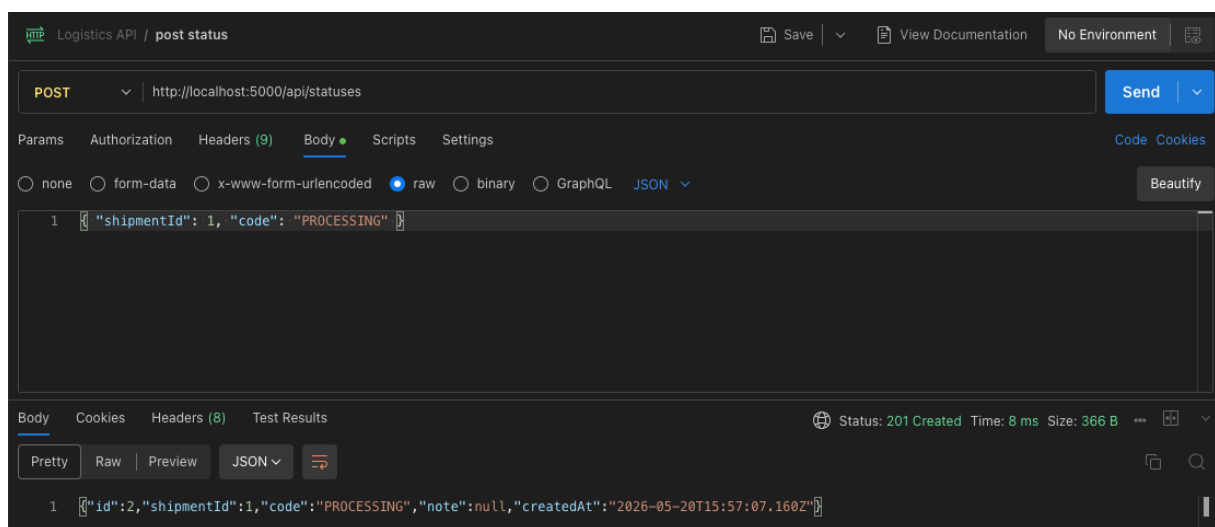


Рис. 5. POST-запит для статусу. Перехід PENDING -> PROCESSING дозволено

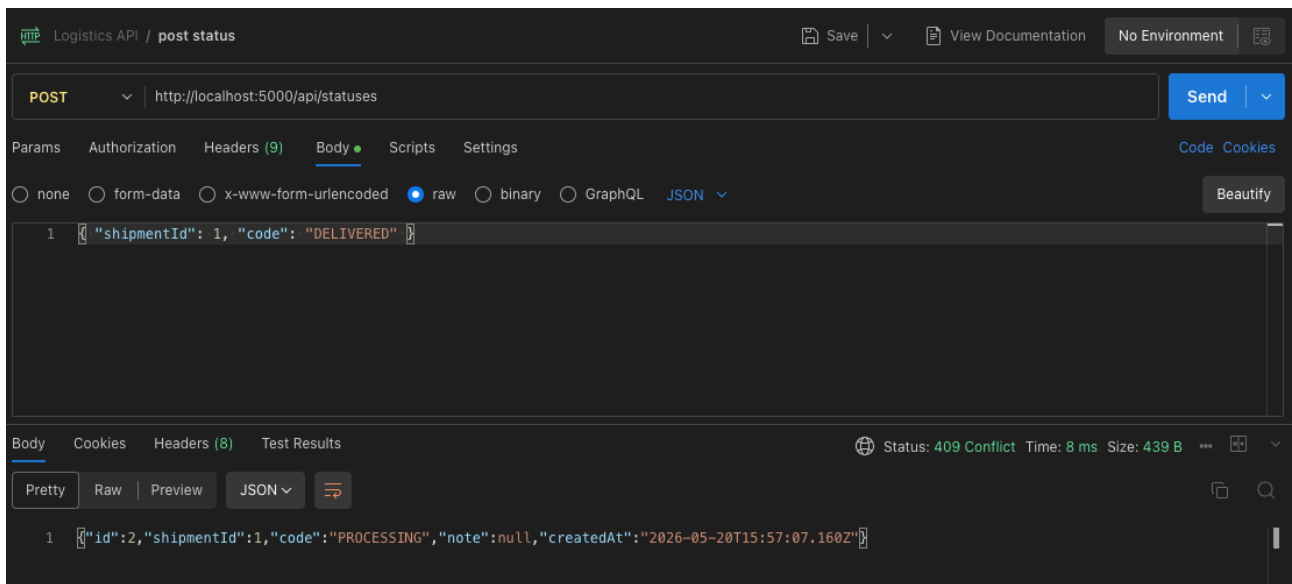


Рис. 6. POST-запит для статусу. Перехід PROCESSING -> DELIVERED не дозволено

Тестовий сценарій №4: Склади та Трансфер

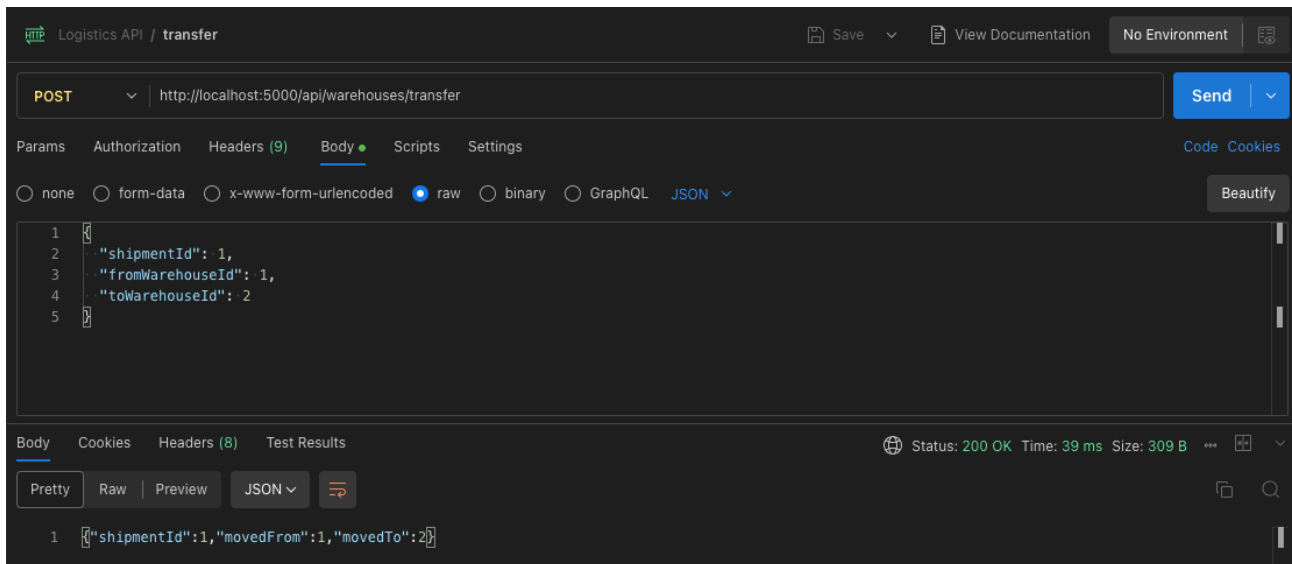


Рис. 7. POST-запит для переміщення товару між складами

Висновок:

Під час виконання цієї лабораторної роботи я сформував базову доменну модель та API, які можуть бути масштабовані.

Логічність: Ендпоінти передбачувані.

Чистота: Використання правильних кодів помилок (400, 404, 409).

Стандарт: Використання JSON як єдиного формату обміну даними.